

DOCUMENT NAME

Document title	Code Standard		
Author(s)	Cuong, Truong Manh		
Role	Team Members		
Date	10/12/2025	File Name:	C1SE.40_CodeStandard_v2.0.docx

REVISION HISTORY

Version	Date	Comments	Author	Approval
1.0	10/10/2025	Initial release	All members	
1.1	22/11/2025	Updated	All members	
2.0	10/12/2025	Official	All members	

TABLE OF CONTENTS

1. Introduction	1
1.1 Purpose	1
1.2 Scope	1
2. Code Standards	1
2.1 Variables.....	1
2.2 Spaces Around Operators	1
2.3 Statement Rules	1
2.4 Object Rules	1
2.5 Line Length	2
2.6 Spaces	2
2.6 Quotes.....	2
2.7 Comparative math	2
2.8 Dot location	2
2.9 Array.....	2
2.10 Modules	2
2.11 Functions	2
2.12 String	3
2.13 Error catching	3
2.14 Files	3
2.15 Others	3

1. Introduction

1.1 Purpose

- This Coding Standard requires certain practices for developing programs in the JavaScript/TypeScript language. The objective of this coding standard is to have a positive effect on
 - + Avoidance of errors/bugs, especially the hard-to-find ones.
 - + Maintainability, by promoting some proven design principles

1.2 Scope

- This standard pertains to the use of the JavaScript/ TypeScript language.

2. Code Standards

2.1 Variables

- Using camelCase for identifier names (variables and functions).
- All names start with a letter.
- Constants (like PI) written in UPPERCASE
- No unused variables.
- For var declarations, write each declaration in its own statement.
- Avoid modifying variables of class declarations.
- Avoid modifying variables declared using const.
- No re-declaring variables.
- Avoid assigning a variable to itself.
- Avoid comparing a variable to itself.
- Restricted names should not be shadowed.

2.2 Spaces Around Operators

- Always put spaces around operators (= + - * /), and after commas.

2.3 Statement Rules

- Put the opening bracket at the end of the first line.
- Use one space before the opening bracket
- Put the closing bracket on a new line, without leading spaces.
- Keep else statements on the same line as their curly braces.

2.4 Object Rules

- Place the opening bracket on the same line as the object name.
- Use colon plus one space between each property and its value

- Do not add a comma after the last property-value pair.
- Place the closing bracket on a new line, without leading spaces.
- Maintain consistency of newlines between object properties.
- Always end an object definition with a semicolon.

2.5 Line Length

- For readability, avoid lines longer than 80 characters

2.6 Spaces

- Use 2 spaces for indentation.
- Add a space after keywords.
- Add a space before a function declaration's parentheses
- Commas should have a space after them.
- Add spaces inside single line blocks.
- No space between function identifiers and their invocations.
- Add space between colon and value in key value pairs.

2.6 Quotes

- Use single quotes for strings except to avoid escaping.

2.7 Comparative math

- Exception: `obj == null` is allowed to check for null || undefined.

2.8 Dot location

- Dot should be on the same line as property.

2.9 Array

- Use array literals instead of array constructors

2.10 Modules

- Use a single import statement per module.
- Renaming import, export, and destructuring assignments to the same name are not allowed.

2.11 Functions

- Avoid unnecessary function binding.
- No unnecessary parentheses around function expressions.
- No function declarations in nested blocks.

2.12 String

- Regular strings must not contain template literal placeholders.
- No octal escape sequences in string literals.
- No multiline strings.
- No spacing in template strings.

2.13 Error catching

- Only throw an Error object.

2.14 Files

- Files must end with a newline.

2.15 Others

- Semicolons must have a space after and no space before.
- Must have a space before blocks.
- Use isNaN() when checking for NaN
- Function typeof must be compared to a valid string.
- Never start a line with (, [, `, or a handful of other unlikely possibilities.